数据结构与算法 A

2022 年期中 · 参考答案

仓库配套答案为个人解答，原文注明“不一定对”。本版重新计算并补全遗漏题；KMP 数组以 0 下标、next[0]=-1 为准。算法给出可核查的思路及伪代码，不作为官方评分标准。

一、选择填空

题号	答案
1	C
2	B
3	B
4	B
5	B、C、D；42 种
6	14 种；{12,24,33,65,33,56,48,92,86,70}
7	C，8 个

2．内层执行 $\lceil \frac{n}{i} \rceil$ 次，总次数介于 $n\sum_{i=1}^{n}\frac{1}{i}$ 与该值加 n 之间，故为 $\Theta(n\log n)$。

4．B 在执行 p->next=s 后，s->next=p->next 令 s 指向自己，丢失原后继。

5．A 可行。B 的 c 出栈后，b 仍在 a 上，不能立即输出 a；C 的 d、e 出栈后，c、b、a 仍按此顺序受栈约束，不能 c 后直接 a；D 同理。可行序列数为 Catalan 数 $C_5=\frac{\binom{10}{5}}{6}=42$。

6．四个互异键的二叉搜索树数为 $C_4=\frac{\binom{8}{4}}{5}=14$。

建最小堆时，只有下标 1 的下沉改变数组：70 与 24 交换，再与末尾 33 交换，得到给定结果。题干原来就有两个 33，应保留，不是识别错误。

7．树的叶子数公式 $n_0=1+\sum_{d\ge1}(d-1)n_d$，得 $1+2+2+3=8$。

二、辨析与简答

1．两种栈的 push/pop 均可在 $O(1)$ 时间完成（定长数组须容量足够；链式实现把栈顶放在链表头）。顺序栈连续存储、指针开销少、局部性好，但容量固定可能浪费或不足。链式栈按需分配，但每结点多一个指针，且有分配管理开销。不能笼统说链式栈总更省空间。

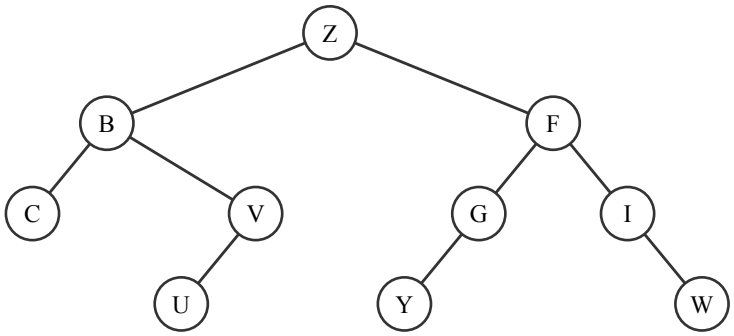
2．（1）全部 18 个不同非空子串，字典序为：

```
1, 12, 121, 1212, 12121, 121212, 1212123,
12123, 123,
2, 21, 212, 2121, 21212, 212123, 2123, 23,
3
```

（2）普通 next=`[-1,0,0,1,2,3,4]`；优化 nextval=`[-1,0,-1,0,-1,0,4]`。

3．缺失字符按题干各空先后分别为：前序 B、G、I；中序 B、Y、W；后序 U、V、F、Z。

```
前序: Z B C V U F G Y I W
中序: C B U V Z Y G F I W
后序: C U V B Y G W I F Z
```



4．合并权重依次为 5+9=14，12+13=25，14+16=30，25+30=55，45+55=100。以下编码为一种合法解：

```
          100
        /     \
     f:45      55
             /    \
           25      30
          / \     /  \
       c:12 d:13 14 e:16
                / \
             a:5 b:9
```

字符	a	b	c	d	e	f
编码	1100	1101	100	101	111	0

平均长度为 $\frac{5\times4+9\times4+12\times3+13\times3+16\times3+45}{100}=2.24$ bit/字符。
6 个字母等长编码需要 3 bit/字符，节省比例 $\frac{3-2.24}{3}=25.333...\%$。交换某内部结点的左右孩子会改变码字，但平均长度不变。

5．（1）森林如下：

```
   1          4          8
  / \        / \         |
 0   3      5   6        7
     |
     2
```

（2）find(2) 压缩 2→3→1 为 2→1；find(7) 经过 7→8。大小为 2 的树 8 并入大小为 4 的树 1。结果：

```
[1,-6,1,1,-3,4,4,8,1]
```

7 仍指向 8，因为 find(7) 发生于 8 被挂到 1 之前；不能未经额外操作就把 7 改指 1。

三、算法填空

```
// (1)
!st.empty()
// (2)
st.pop()
// (3)
topElem.rd == 1
// (4)
topElem.node->getChildrenNum() - i - 1
// (5)
stackElement(child, 0)
```

先压入根的“待访问”标记，再从右到左压入孩子，确保出栈顺序是从左到右。每个结点至多压栈两次，总时间 $O(n)$，最坏栈空间 $O(n)$。第 4 空不能直接填 i，否则兄弟结点访问次序会反转。

四、算法设计与实现

1．保持 A 的队首始终为栈顶，B 在每次操作后为空。

```
push(x):
    B.push_back(x)
    while not A.empty():
        B.push_back(A.front())
        A.pop_front()
    while not B.empty():
        A.push_back(B.front())
        B.pop_front()

top():   return A.front()
pop():   A.pop_front()
empty(): return A.empty()
```

push 后新元素位于队首，原栈元素的相对顺序保持不变，故满足后进先出。push 为 $O(n)$，其余 $O(1)$；除题目提供的两个队列，只使用常数个临时变量。

2．沿“两个等长相邻块的间距”枚举，使用最长公共前缀的逆向递推，时间 $O(n^2)$、额外空间 $O(1)$。

```
answer = n
for d = 1..floor(n/2):
    common = 0
    for i = n-d-1 down to 0:
        if S[i] == S[i+d]:
            common += 1
        else:
            common = 0
        if common >= d:
            answer = min(answer, i)
return -1 if answer == n else answer
```

对固定 d，递推始终使 common 等于 LCP(S[i..], S[i+d..])。当 common $\geqslant$ d 时，S[i..i+d-1] 与紧邻的 S[i+d..i+2d-1] 相同，恰为 AA。反之，任何 AA 都会在其半长 d 与起点 i 被检测到。枚举全部 d，并取最小 i，得到首次起始下标。每个 d 扫描 $O(n)$ 个位置，总计 $O(n^2)$。

3. 用根到当前结点路径上的前缀和频数表。到当前结点前缀和为 s，则以当前结点结束、和为 targetSum 的路径数为此前前缀和 s-targetSum 的出现次数。

```
count = {0: 1}
DFS(node, prefix):
    if node is null: return 0
    s = prefix + node.value
    result = count[s-targetSum]  // Missing keys have count zero.
    count[s] += 1
    result += DFS(node.left, s)
    result += DFS(node.right, s)
    count[s] -= 1              // Remove this node before the next branch.
    return result
answer = DFS(root, 0)
```

频数表始终只表示当前根路径上的祖先前缀，回溯时减一，防止把不同分支拼接成路径。每条向下路径由一对祖先前缀唯一确定，因此不重不漏。

哈希表操作期望 $O(1)$ 时，总时间期望 $O(n)$，额外空间 $O(h)$（最坏 $O(n)$）。若使用平衡搜索树，可保证 $O(n\log n)$ 时间。题目不要求最优时，也可从每结点向下枚举，最坏 $O(n^2)$。样例三条为 5→3、5→2→1、-3→11。

五、分析证明

1. （1）记叶子数为 L。所有内部结点贡献 km 条边；树边数也等于结点数减一，即 m+L-1。故 $km = m + L - 1$，得到

$$L = (k-1)m + 1.$$

（2）高度 h 时，每层至多有 k^{i-1} 个结点，故最大为

$$1 + k + \ldots + k^{h-1} = \frac{k^h - 1}{k - 1}.$$

取每层全部结点都具有 k 个孩子的树可达到上界。

至少有一条含 h 个结点的根叶路径，其前 h-1 个结点都是内部结点，因此 m⩾h-1。结合总结点数 $N = km + 1$，有

$$N \geq k(h-1) + 1.$$

构造“主链上的每个内部结点有一个继续向下的孩子，其余 k-1 个孩子全为叶子”，可达到下界。h=1 时上下界均为 1。